Mills Museum AI Keyword Generation System

Abstract

This project presents an end-to-end system for automating keyword annotation of museum artworks using the Getty Art and Architecture Thesaurus. The system integrates data engineering, semantic embeddings, approximate nearest neighbor search, clustering, and a human-in-the-loop user interface. Emphasis is placed on data quality, reproducibility, and extensibility, so future contributors can onboard quickly and improve components independently.

Link to video explanation can be found here: https://northeastern-my.sharepoint.com/:f:/g/personal/manakelew_a_northeastern_edu/IgBmkNxelnMkQpnTRdHTRu2xAZKHKzEZvTImH3_wgGvFEB0?e=xA9aFq

1. System Overview

The project is organized as a modular pipeline that transforms vocabulary data and museum images into validated keyword outputs. Art and Architecture Thesaurus data is first cleaned and deduplicated to form a standardized vocabulary. After that, the museum images are processed by the keyword generation model, matched to the vocabulary in the Thesaurus used, and then reviewed by the user in the user interface before the final result is exported.

The system is built around three design principles: separation of concerns between data, model, and interface layers; deterministic preprocessing for reproducibility; and human-in-the-loop validation for quality control.

Pipeline Diagram

```
AAT Data -> Cleaning -> Deduplication -> Clean Vocabulary
                                          |
                                          v
Images -> Embedding Model -> Raw Keywords -> AAT Matching ->
User Interface
```

2. Setup and Prerequisites

To run the backend locally, contributors need Python 3.10 or later and libraries such as datasets, pandas, pyarrow, sentence-transformers, Facebook AI Similarity Search, numpy, and torch. To run the frontend locally, contributors need Node.js and npm for the React-based user interface.

A typical local workflow is to load the dataset, run the semantic deduplication pipeline, prepare museum data, and then launch the frontend. Running the pipeline in this order makes sure that cached data, cleaned vocabularies, and planned intermediate outputs exist before a contributor starts testing the user interface.

```
python load_dataset.py
 python semantic_dedup_pipeline.py
 python data_to_hf.py
 # then launch the frontend development server
```

3. Backend Architecture

3.1 Data Layer

The data layer is in charge of loading, caching, and converting the Art and Architecture Thesaurus and museum metadata into types suitable for analysis and modeling.

load_dataset.py

The Art and Architecture Thesaurus dataset is downloaded by this module from Hugging Face and saved locally as a parquet file. Parquet was chosen over plain-text formats like comma-separated values files because it stores data more effectively and completes input and output tasks more quickly.

File: load_dataset.py

```
from datasets import load_dataset

dataset = load_dataset("aat_museum_subset")
 df = dataset["train"].to_pandas()
 df.to_parquet("data_cache/aat.parquet")
```

Alternative considered: comma-separated values files. This was not selected because plain-text parsing is slower, file sizes are larger, and repeated development iterations become less efficient.

data_to_hf.py

This module converts museum metadata into a machine learning-ready dataset and links museum images to records using accession numbers. A flexible file-matching strategy was selected because museum image naming is not always perfectly uniform.

File: data_to_hf.py

```
# Match images flexibly using accession number prefixes
 matches = glob.glob(f"data/images/{accession_number}*.png")
```

Alternative considered: enforcing a strict naming convention. This was not selected because real museum collections often contain inconsistent file naming, and a rigid approach would create fragile preprocessing.

3.2 Semantic Deduplication Pipeline

The semantic deduplication pipeline is the core backend contribution of the project. Its purpose is to reduce redundant or near-duplicate controlled vocabulary labels so downstream matching becomes more consistent and less noisy.

Step 1: Language Normalization

Language normalization ensures that missing or null language identifiers are handled consistently and that semantically similar labels are not incorrectly merged across languages.

File: semantic_dedup_core.py

```
# Prevent cross-language merging by normalizing missing language
values
 LANG_NULL_TOKEN = "__NULL_LANGUAGE__"
 df["language_id"] = df["language_id"].fillna(LANG_NULL_TOKEN)
```

Step 2: Embedding Generation

Each label is converted into a dense vector embedding using a SentenceTransformers model. The model all-MiniLM-L6-v2 was selected because it offers a good balance between speed and semantic quality for a project of this scale.

File: semantic_dedup_core.py

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")

# normalize_embeddings=True allows cosine similarity to be
```

```
computed with dot product
 embeddings = model.encode(labels, normalize_embeddings=True)
```

Alternative considered: larger transformer models such as MPNet. This was not selected because the latency and computing costs were higher than necessary for the size of this dataset.

Step 3: Similarity Search

The project uses Facebook AI Similarity Search to find nearest neighbors efficiently in embedding space. Because embeddings are normalized, inner product search corresponds to cosine similarity.

File: semantic_dedup_core.py

```
import faiss

index = faiss.IndexFlatIP(embeddings.shape[1])
 index.add(embeddings)

# Find nearest neighbors for each label
 sims, neighbors = index.search(embeddings, k=10)
```

Alternative considered: brute-force pairwise search. This was not selected because it scales poorly and becomes computationally expensive as the number of labels grows.

Step 4: Clustering

Similarity edges above a threshold are merged using a Union-Find data structure. This supports flexible connected-component clustering without requiring a fixed number of clusters.

File: semantic_dedup_core.py

```
# Merge labels into the same cluster if similarity exceeds the
threshold
 if sims[i, j] >= threshold:
     uf.union(i, j)
```

Alternative considered: K-Means clustering. This was not selected because the number of clusters is not known in advance and cluster sizes are naturally uneven. Alternative considered:

hierarchical clustering. This was not selected because it is more expensive in time and memory for this use case.

Step 5: Canonical Label Selection

After clusters are formed, the system selects a canonical label deterministically. This matters because clustering alone is not sufficient; each group still needs a representative label for downstream use.

File: semantic_dedup_core.py

```
# Select canonical label from a cluster
 canonical = sorted(cluster, key=lambda x: (
     not has_note[x],   # prefer labels with descriptions
     len(x),            # prefer shorter labels
     x                  # alphabetical fallback
 ))[0]
```

This strategy prioritizes labels with descriptions, then shorter labels, then alphabetical order. The goal is to balance informativeness, readability, and determinism.

Step 6: Metrics and Outputs

The pipeline evaluates quality using semantic collision rate and token entropy. It also writes multiple files so contributors can inspect both the final vocabulary and the transformation process.

File: semantic_dedup_core.py

```
deduped.to_parquet(args.output_parquet, index=False)
 audit_df.to_csv(args.audit_csv, index=False)
 mapping_df.to_csv(args.mapping_csv, index=False)

print(f"Saved deduplicated parquet: {args.output_parquet}")
 print(f"Saved audit CSV: {args.audit_csv}")
 print(f"Saved mapping CSV: {args.mapping_csv}")
```

Deduplication Workflow Diagram

```
Labels -> Embeddings -> Similarity Search -> Filter by Threshold
-> Clustering -> Canonical Selection -> Clean Vocabulary
```

4. Backend and Application Programming Interface

The backend supports both the **data-processing pipeline** and the **keyword-generation interface**. Loading and preparing controlled vocabulary data, deduplicating semantically related labels, providing keyword predictions for uploaded images, and returning formatted results that the frontend can present and export are its four main tasks. The primary backend data-cleaning step is still the semantic deduplication pipeline. It creates three outputs including a cleaned parquet file, an audit comma-separated values file, and a term-level mapping comma-separated values file. It also embeds labels, clusters semantically related terms within each language, and selects consistent labels. More detailed internal documentation, such as comments outlining dependency checks, vector normalization, language isolation, cluster previews, and the rationale behind the creation of numerous output files for various audiences, is also included.

The deduplication stage's command-line entrypoint is purposefully short. The primary implementation can be imported for testing and reuse, but the wrapper script allows contributors to run the pipeline directly. Because its fundamental features are modular and the executable script is kept simple, this approach enhances maintainability.

The backend provides a POST /predict application programming interface endpoint for interactive keyword creation. This endpoint receives uploaded photos from the React/Vite browser application via Hypertext Transfer Protocol, and shared adapters in the frontend utility layer transform the response into keyword objects that are easy to use. Downstream review actions like keyword inclusion or exclusion, export to text or comma-separated values, and confidence display are supported by the backend output.

File: src/frontend/web/src/App.jsx

```
const formData = new FormData()
formData.append('file', file)
formData.append('term_count', String(termCount))

const res = await fetch(`${API_URL}/predict`, {
     method: 'POST',
     headers: { 'ngrok-skip-browser-warning': 'true' },
     body: formData,
})

const data = await res.json()
```

A typical response is a JavaScript Object Notation object containing a list of keyword predictions and confidence values. Error handling should return appropriate status codes for invalid files, missing inputs, or model failures.

```
{
  "keywords": [
    { "text": "historic landscapes", "confidence": 51.8 },
    { "text": "atmospheric perspective", "confidence": 51.4 },
    { "text": "landscapists", "confidence": 51.0 }
  ]
}
```

5. Frontend Architecture

The frontend is now more advanced than the earlier upload-process-review description and is now a more advanced (hierarchies, description, etc.) keyword review environment rather than only a simple result screen. The repository currently contains two user-facing keyword-review implementations: a **Python/Gradio workflow** in src/frontend/gradio.py and a **React/Vite browser application** in src/frontend/web/. There is also a third interface, the Gradio-based data exhibit in src/frontend/data_viz/data_story_exhibit.py, but that tool is for exploring the Art and Architecture Thesaurus dataset rather than reviewing image keywords.

The React/Vite application follows the same broad product flow, which is to upload, process, and review, but the review stage now includes several added controls and visualization options. The updated user interface shows a slider for the number of keywords to gen**erate**, a **query bias** control that balances image information against description information, a **diversity (Maximal Marginal Relevance)** control that adjusts the tradeoff between diversity and pure relevance, a visible **Artificial Intelligence description** panel that describes the image during processing, and review tools that support **grouped or hierarchy-based presentation**, **heatmap view**, filtering, copying, and multiple export formats.

Query Bias

This setting controls how much weight is given to the uploaded image versus the generated or provided description when producing keyword suggestions. A value near zero favors image evidence, while a value near one favor descriptive text. The default centered value reflects a balanced query.

Diversity (Maximal Marginal Relevance)

This setting controls whether the output list should prioritize a broader range of keywords or concentrate on highly similar, highly relevant terms. A lower value encourages variety, while a higher value pushes the results toward pure relevance. The user interface currently shows this as a slider from “Diverse” to “Relevant,” with a default close to relevance-heavy output.

Artificial Intelligence Description

During the processing stage, the interface surfaces with an Artificial Intelligence-generated textual description of the artwork while processing the images. This is important because it gives the user a readable explanation of what the system is understanding about the image before the final keyword list is shown. The processing screen directly shows a description card next to the image preview.

Grouped/Hierarchy-Based Review

The review screen supports a grouped layout where suggested keywords are organized under higher-level hierarchies such as Built Environment, Color, Components, etc. This makes the output easier to inspect because related terms are visually clustered rather than shown only as one flat list. The screenshots clearly show grouped keyword sections in the updated interface.

Heatmap view

The review interface also includes a heatmap mode, which highlights the words that are the most related to the image (highest confidence score) so the user can see it easily, and thus make the final review process more accurate.

Frontend Workflow Diagram

```
Upload → Configure generation settings → Process image

       → View Artificial Intelligence description

       → Review keywords in flat, grouped, or heatmap-oriented
     views

       → Filter, include or exclude, copy, or export
```

To run the frontend locally, user should go to the frontend directory, install dependencies with npm install, and start the development server.

6. Visualization Layer

The visualization layer provides information about the system's behavior using techniques such as Principal Component Analysis, heatmaps, word clouds, and network graphs. These visualizations help identify clustering quality, keyword relationships, and potential dataset biases.

Implementation Examples

File: 06_advanced_analysis.py

```
from sklearn.decomposition import PCA
 import matplotlib.pyplot as plt

pca = PCA(n_components=2)
 points_2d = pca.fit_transform(scope_matrix.toarray())

plt.scatter(points_2d[:, 0], points_2d[:, 1], s=8, alpha=0.5)
 plt.xlabel("Principal Component 1")
 plt.ylabel("Principal Component 2")
 plt.title("PCA Projection of Scope Note Embeddings")
 plt.show()
```

File: 04_visualizations.py

```
import seaborn as sns
 import matplotlib.pyplot as plt

sns.heatmap(heatmap_df, annot=True, cmap="YlOrRd")
 plt.title("Century References by Facet")
 plt.xlabel("Century")
 plt.ylabel("Facet")
 plt.show()
```

Visualization Workflow Diagram

```
Dataset -> Feature Extraction -> Dimensionality Reduction ->
Visualization
                                                |-> Principal
Component Analysis
                                                |-> Heatmap
                                                |-> Word Cloud
                                                |-> Network Graph
```

7. Onboarding and Expected Outputs

There are specific steps that should be followed when onboarding new participants. At each stage, the user should check that certain files/interface outputs are created. This reduces debugging time and helps isolate configuration problems early.

Verification Examples

File: load_dataset.py

```
df = load_aat_dataset()
 print(f"Shape: {df.shape}")
 print(f"Columns: {list(df.columns)}")
```

Expected output: a dataset structure and a list of expected schema columns.

File: semantic_dedup_core.py

```
print(f"Saved deduplicated parquet: {args.output_parquet}")
 print(f"Saved audit CSV: {args.audit_csv}")
 print(f"Saved mapping CSV: {args.mapping_csv}")
```

Expected output: verification that deduplicated parquet, audit comma-separated values file, and mapping comma-separated values file were created.

File: frontend keywordAdapters.js

```
console.log(data.keywords)
```

```
{
   "keywords": [
     { "text": "modern", "confidence": 97 },
     { "text": "sculpture", "confidence": 92 }
   ]
 }
```

Onboarding Workflow Diagram

```
Load Dataset -> Verify Parquet
        |
        v
 Run Deduplication -> Verify Deduplicated Outputs
        |
        v
 Prepare Museum Data -> Verify Clean Dataset
        |
        v
 Run Frontend -> Verify Keywords Appear in User Interface
```

8. Documentation Consistency and Clarity

Both terminology and code presentation in documentation should be consistent. For instance, if the phrase "user interface" is used once then it should be used repeatedly. Code snippets should always display the file location, provide comments, and display the predicted results.

File: semantic_dedup_core.py

```
# Normalize missing language values so clustering remains
language-aware
 LANG_NULL_TOKEN = "__NULL_LANGUAGE__"
 df["language_id"] = df["language_id"].fillna(LANG_NULL_TOKEN)
```

9. Design Decisions and Alternatives Considered

Many implementation choices were taken to maintain a balance between maintainability, scalability, speed, and clarity.

Larger models were not chosen for embeddings because of unnecessary lag time, but MiniLM was chosen since it is quick and effective. The team decided to use the Union-Find clustering approach since it allows for flexible cluster sizes and doesn't require predefining the number of clusters. We also looked at K-Means and hierarchical clustering but decided not to use them since our data didn't fit the data structure or performance requirements.

10. Limitations

The system still has important limitations. Some model-generated keywords may be inaccurate. Additionally, some of the descriptions generated by the keywords (dropdown) are also not very accurate. In addition, embeddings are not yet fine-tuned specifically on the Art and Architecture Thesaurus domain.

11. Future Work

Future improvements could include fine-tuning embeddings on the Art and Architecture Thesaurus collection, integrating stronger reranking models, adding multimodal image-and-text matching, and creating a feedback loop so user corrections (report an error) improve future predictions.

12. How to Onboard and Contribute

A new user should begin by reading the backend architecture section, then studying semantic_dedup_core.py because it contains the core algorithmic logic. From there, they should run the project by loading the dataset, running deduplication, preparing museum data, and then testing the frontend.

The most important starting points are semantic_dedup_core.py for core logic, data_to_hf.py for data preparation, and the frontend directory for interface behavior. Users who want to improve the model should start in the deduplication and matching pipeline; users who want to improve usability should begin in the React components.

Common errors include incorrect image-to-accession matching, low-confidence keyword predictions, and incorrect clustering caused by inconsistent language normalization.

13. Conclusion

This system shows how dictionary keywords, architecture thesaurus, and artificial intelligence can be used to automate complex museum tagging workflows. The modular architecture allows the different components to be improved independently with human-in-the-loop validation to

preserve interpretability and quality. As a result, the project provides both a practical tool for museum workflows and a strong foundation for future contributors.